```
DX::ThrowIfFailed(
    MFCreateSourceReaderFromURL(url, nullptr, &m_reader)
    );
```

The **MediaStreamer::Initialize** method then creates an IMFMediaType object using MFCreateMediaType to describe the format of the audio stream. An audio format has two types: a major type and a subtype. The major type defines the overall format of the media, such as video, audio, script, and so on. The subtype defines the format, such as PCM, ADPCM, or WMA.

The **MediaStreamer::Initialize** method uses the IMFAttributes::SetGUID method to specify the major type (MF_MT_MAJOR_TYPE) as audio (**MFMediaType_Audio**) and the minor type (MF_MT_SUBTYPE) as uncompressed PCM audio (**MFAudioFormat_PCM**). **MF_MT_MAJOR_TYPE** and **MF_MT_SUBTYPE** are Media Foundation Attributes. **MFMediaType_Audio** and **MFAudioFormat_PCM** are type and subtype GUIDs; see Audio Media Types for more information. The IMFSourceReader::SetCurrentMediaType method associates the media type with the stream reader.

C++

```
// Set the decoded output format as PCM.
// XAudio2 on Windows can process PCM and ADPCM-encoded buffers.
// When this sample uses Media Foundation, it always decodes into PCM.

DX::ThrowIfFailed(
    MFCreateMediaType(&mediaType)
    );

DX::ThrowIfFailed(
    mediaType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Audio)
    );

DX::ThrowIfFailed(
    mediaType->SetGUID(MF_MT_SUBTYPE, MFAudioFormat_PCM)
    );

DX::ThrowIfFailed(
    m_reader->SetCurrentMediaType(MF_SOURCE_READER_FIRST_AUDIO_STREAM, 0,
mediaType.Get())
    );
```

The **MediaStreamer::Initialize** method then obtains the complete output media format from Media Foundation using IMFSourceReader::GetCurrentMediaType and calls the MFCreateWaveFormatExFromMFMediaType method to convert the Media Foundation audio media type to a WAVEFORMATEX structure. The **WAVEFORMATEX** structure

defines the format of waveform-audio data. Marble Maze uses this structure to create the source voices and to apply the low-pass filter to the marble rolling sound.

```
C++
```

```
// Get the complete WAVEFORMAT from the Media Type.
DX::ThrowIfFailed(
    m_reader->GetCurrentMediaType(MF_SOURCE_READER_FIRST_AUDIO_STREAM,
&outputMediaType)
    );

uint32 formatSize = 0;
WAVEFORMATEX* waveFormat;
DX::ThrowIfFailed(
    MFCreateWaveFormatExFromMFMediaType(outputMediaType.Get(), &waveFormat,
&formatSize)
    );
CopyMemory(&m_waveFormat, waveFormat, sizeof(m_waveFormat));
CoTaskMemFree(waveFormat);
```

> **ⓘ Important**
>
> The MFCreateWaveFormatExFromMFMediaType method uses **CoTaskMemAlloc** to allocate the WAVEFORMATEX object. Therefore, make sure that you call **CoTaskMemFree** when you are finished using this object.

The **MediaStreamer::Initialize** method finishes by computing the length of the stream, **m_maxStreamLengthInBytes**, in bytes. To do so, it calls the IMFSourceReader::GetPresentationAttribute method to get the duration of the audio stream in 100-nanosecond units, converts the duration to sections, and then multiplies by the average data transfer rate in bytes per second. Marble Maze later uses this value to allocate the buffer that holds each gameplay sound.

```
C++
```

```
// Get the total length of the stream, in bytes.
PROPVARIANT var;
DX::ThrowIfFailed(
    m_reader->
        GetPresentationAttribute(MF_SOURCE_READER_MEDIASOURCE,
MF_PD_DURATION, &var)
    );

// duration is in 100ns units; convert to seconds, and round up
// to the nearest whole byte.
```

```
ULONGLONG duration = var.uhVal.QuadPart;
m_maxStreamLengthInBytes =
    static_cast<unsigned int>(
        ((duration * static_cast<ULONGLONG>(m_waveFormat.nAvgBytesPerSec)) +
10000000)
        / 10000000
        );
```

Creating the source voices

Marble Maze creates XAudio2 source voices to play each of its game sounds and music in source voices. The **Audio** class defines an IXAudio2SourceVoice object for the background music and an array of **SoundEffectData** objects to hold the gameplay sounds. The **SoundEffectData** structure holds the **IXAudio2SourceVoice** object for an effect and also defines other effect-related data, such as the audio buffer. **Audio.h** defines the **SoundEvent** enumeration. Marble Maze uses this enumeration to identify each gameplay sound. The **Audio** class also uses this enumeration to index the array of **SoundEffectData** objects.

C++

```
enum SoundEvent
{
    RollingEvent        = 0,
    FallingEvent        = 1,
    CollisionEvent      = 2,
    CheckpointEvent     = 3,
    MenuChangeEvent     = 4,
    MenuSelectedEvent   = 5,
    LastSoundEvent,
};
```

The following table shows the relationship between each of these values, the file that contains the associated sound data, and a brief description of what each sound represents. The audio files are located in the **\Media\Audio** folder.

Expand table

SoundEvent value	File name	Description
RollingEvent	MarbleRoll.wav	Played as the marble rolls.
FallingEvent	MarbleFall.wav	Played when the marble falls off the maze.
CollisionEvent	MarbleHit.wav	Played when the marble collides with the maze.

SoundEvent value	File name	Description
CheckpointEvent	Checkpoint.wav	Played when the marble passes over a checkpoint.
MenuChangeEvent	MenuChange.wav	Played when the user changes the current menu item.
MenuSelectedEvent	MenuSelect.wav	Played when the user selects a menu item.

The following example shows how the **Audio::CreateResources** method creates the source voice for the background music. The XAUDIO2_SEND_DESCRIPTOR structure defines the target destination voice from another voice and specifies whether a filter should be used. Marble Maze calls the **Audio::SetSoundEffectFilter** method to use the filters to change the sound of the ball as it rolls. The XAUDIO2_VOICE_SENDS structure defines the set of voices to receive data from a single output voice. Marble Maze sends data from the source voice to the mastering voice (for the dry, or unaltered, portion of a playing sound) and to the two submix voices that implement the wet, or reverberant, portion of a playing sound.

The IXAudio2::CreateSourceVoice method creates and configures a source voice. It takes a WAVEFORMATEX structure that defines the format of the audio buffers that are sent to the voice. As mentioned previously, Marble Maze uses the PCM format.

C++

```
XAUDIO2_SEND_DESCRIPTOR descriptors[3];
descriptors[0].pOutputVoice = m_musicMasteringVoice;
descriptors[0].Flags = 0;
descriptors[1].pOutputVoice = m_musicReverbVoiceSmallRoom;
descriptors[1].Flags = 0;
descriptors[2].pOutputVoice = m_musicReverbVoiceLargeRoom;
descriptors[2].Flags = 0;
XAUDIO2_VOICE_SENDS sends = {0};
sends.SendCount = 3;
sends.pSends = descriptors;
WAVEFORMATEX& waveFormat = m_musicStreamer.GetOutputWaveFormatEx();

DX::ThrowIfFailed(
    m_musicEngine->CreateSourceVoice(&m_musicSourceVoice, &waveFormat, 0,
1.0f, &m_voiceContext, &sends, nullptr)
    );

DX::ThrowIfFailed(
    m_musicMasteringVoice->SetVolume(0.4f)
    );
```

Playing background music

A source voice is created in the stopped state. Marble Maze starts the background music in the game loop. The first call to **MarbleMazeMain::Update** calls **Audio::Start** to start the background music.

```
C++
```

```
if (!m_audio.m_isAudioStarted)
{
    m_audio.Start();
}
```

The **Audio::Start** method calls IXAudio2SourceVoice::Start to start to process the source voice for the background music.

```
C++
```

```
void Audio::Start()
{
    if (m_engineExperiencedCriticalError)
    {
        return;
    }

    HRESULT hr = m_musicSourceVoice->Start(0);

    if SUCCEEDED(hr) {
        m_isAudioStarted = true;
    }
    else
    {
        m_engineExperiencedCriticalError = true;
    }
}
```

The source voice passes that audio data to the next stage of the audio graph. In the case of Marble Maze, the next stage contains two submix voices that apply the two reverb effects to the audio. One submix voice applies a close late-field reverb; the second applies a far late-field reverb.

The amount that each submix voice contributes to the final mix is determined by the size and shape of the room. The near-field reverb contributes more when the ball is near a wall or in a small room, and the late-field reverb contributes more when the ball is in a large space. This technique produces a more realistic echo effect as the marble moves through the maze. To learn more about how Marble Maze implements this effect, see

Audio::SetRoomSize and **Physics::CalculateCurrentRoomSize** in the Marble Maze source code.

> ⓘ **Note**
>
> In a game in which most room sizes are relatively the same, you can use a more basic reverb model. For example, you can use one reverb setting for all rooms or you can create a predefined reverb setting for each room.

The **Audio::CreateResources** method uses Media Foundation to load the background music. At this point, however, the source voice does not have audio data to work with. In addition, because the background music loops, the source voice must be regularly updated with data so that the music continues to play.

To keep the source voice filled with data, the game loop updates the audio buffers every frame. The **MarbleMazeMain::Render** method calls **Audio::Render** to process the background music audio buffer. The **Audio** class defines an array of three audio buffers, **m_audioBuffers**. Each buffer holds 64 KB (65536 bytes) of data. The loop reads data from the Media Foundation object and writes that data to the source voice until the source voice has three queued buffers.

> ⊗ **Caution**
>
> Although Marble Maze uses a 64 KB buffer to hold music data, you may need to use a larger or smaller buffer. This amount depends on the requirements of your game.

C++

```
// This sample processes audio buffers during the render cycle of the
application.
// As long as the sample maintains a high-enough frame rate, this approach
should
// not glitch audio. In game code, it is best for audio buffers to be
processed
// on a separate thread that is not synced to the main render loop of the
game.
void Audio::Render()
{
    if (m_engineExperiencedCriticalError)
    {
        m_engineExperiencedCriticalError = false;
        ReleaseResources();
        Initialize();
        CreateResources();
```

```
        Start();
        if (m_engineExperiencedCriticalError)
        {
            return;
        }
    }

    try
    {
        bool streamComplete;
        XAUDIO2_VOICE_STATE state;
        uint32 bufferLength;
        XAUDIO2_BUFFER buf = {0};

        // Use MediaStreamer to stream the buffers.
        m_musicSourceVoice->GetState(&state);
        while (state.BuffersQueued <= MAX_BUFFER_COUNT - 1)
        {
            streamComplete = m_musicStreamer.GetNextBuffer(
                m_audioBuffers[m_currentBuffer],
                STREAMING_BUFFER_SIZE,
                &bufferLength
                );

            if (bufferLength > 0)
            {
                buf.AudioBytes = bufferLength;
                buf.pAudioData = m_audioBuffers[m_currentBuffer];
                buf.Flags = (streamComplete) ? XAUDIO2_END_OF_STREAM : 0;
                buf.pContext = 0;
                DX::ThrowIfFailed(
                    m_musicSourceVoice->SubmitSourceBuffer(&buf)
                    );

                m_currentBuffer++;
                m_currentBuffer %= MAX_BUFFER_COUNT;
            }

            if (streamComplete)
            {
                // Loop the stream.
                m_musicStreamer.Restart();
                break;
            }

            m_musicSourceVoice->GetState(&state);
        }
    }
    catch (...)
    {
        m_engineExperiencedCriticalError = true;
    }
}
```

The loop also handles when the Media Foundation object reaches the end of the stream. In this case, it calls the IMFSourceReader::SetCurrentPosition method to reset the position of the audio source.

```
C++
```

```
void MediaStreamer::Restart()
{
    if (m_reader == nullptr)
    {
        return;
    }

    PROPVARIANT var = {0};
    var.vt = VT_I8;

    DX::ThrowIfFailed(
        m_reader->SetCurrentPosition(GUID_NULL, var)
        );
}
```

To implement audio looping for a single buffer (or for an entire sound that is fully loaded into memory), you can set the XAUDIO2_BUFFER::LoopCount field to **XAUDIO2_LOOP_INFINITE** when you initialize the sound. Marble Maze uses this technique to play the rolling sound for the marble.

```
C++
```

```
if (sound == RollingEvent)
{
    m_soundEffects[sound].m_audioBuffer.LoopCount = XAUDIO2_LOOP_INFINITE;
}
```

However, for the background music, Marble Maze manages the buffers directly so that it can better control the amount of memory that is used. When your music files are large, you can stream the music data into smaller buffers. Doing so can help balance memory size with the frequency of the game's ability to process and stream audio data.

> **Tip**
>
> If your game has a low or varying frame rate, processing audio on the main thread can produce unexpected pauses or pops in the audio because the audio engine has insufficient buffered audio data to work with. If your game is sensitive to this issue, consider processing audio on a separate thread that does not perform rendering.

This approach is especially useful on computers that have multiple processors because your game can use idle processors.

Reacting to game events

The **Audio** class provides methods such as **PlaySoundEffect**, **IsSoundEffectStarted**, **StopSoundEffect**, **SetSoundEffectVolume**, **SetSoundEffectPitch**, and **SetSoundEffectFilter** to enable the game to control when sounds play and stop, and to control sound properties such as volume and pitch. For example, if the marble falls off the maze, **MarbleMazeMain::Update** calls the **Audio::PlaySoundEffect** method to play the **FallingEvent** sound.

C++

```
m_audio.PlaySoundEffect(FallingEvent);
```

The **Audio::PlaySoundEffect** method calls the IXAudio2SourceVoice::Start method to begin playback of the sound. If the **IXAudio2SourceVoice::Start** method has already been called, it is not started again. **Audio::PlaySoundEffect** then performs custom logic for certain sounds.

C++

```
void Audio::PlaySoundEffect(SoundEvent sound)
{
    XAUDIO2_BUFFER buf = {0};
    XAUDIO2_VOICE_STATE state = {0};

    if (m_engineExperiencedCriticalError)
    {
        // If there's an error, then we'll recreate the engine on the next
        // render pass.
        return;
    }

    SoundEffectData* soundEffect = &m_soundEffects[sound];
    HRESULT hr = soundEffect->m_soundEffectSourceVoice->Start();

    if FAILED(hr)
    {
        m_engineExperiencedCriticalError = true;
        return;
    }

    // For one-off voices, submit a new buffer if there's none queued up,
    // and allow up to two collisions to be queued up.
    if (sound != RollingEvent)
```

```
    {
        XAUDIO2_VOICE_STATE state = {0};

        soundEffect->m_soundEffectSourceVoice->
            GetState(&state, XAUDIO2_VOICE_NOSAMPLESPLAYED);

        if (state.BuffersQueued == 0)
        {
            soundEffect->m_soundEffectSourceVoice->
                SubmitSourceBuffer(&soundEffect->m_audioBuffer);
        }
        else if (state.BuffersQueued < 2 && sound == CollisionEvent)
        {
            soundEffect->m_soundEffectSourceVoice->
                SubmitSourceBuffer(&soundEffect->m_audioBuffer);
        }

        // For the menu clicks, we want to stop the voice and replay the
click
        // right away.
        // Note that stopping and then flushing could cause a glitch due to
the
        // waveform not being at a zero-crossing, but due to the nature of
the
        // sound (fast and 'clicky'), we don't mind.
        if (state.BuffersQueued > 0 && sound == MenuChangeEvent)
        {
            soundEffect->m_soundEffectSourceVoice->Stop();
            soundEffect->m_soundEffectSourceVoice->FlushSourceBuffers();

            soundEffect->m_soundEffectSourceVoice->
                SubmitSourceBuffer(&soundEffect->m_audioBuffer);

            soundEffect->m_soundEffectSourceVoice->Start();
        }
    }

    m_soundEffects[sound].m_soundEffectStarted = true;
}
```

For sounds other than rolling, the **Audio::PlaySoundEffect** method calls IXAudio2SourceVoice::GetState to determine the number of buffers that the source voice is playing. It calls IXAudio2SourceVoice::SubmitSourceBuffer to add the audio data for the sound to the voice's input queue if no buffers are active. The **Audio::PlaySoundEffect** method also enables the collision sound to be played two times in sequence. This occurs, for example, when the marble collides with a corner of the maze.

As already described, the Audio class uses the **XAUDIO2_LOOP_INFINITE** flag when it initializes the sound for the rolling event. The sound starts looped playback the first time that **Audio::PlaySoundEffect** is called for this event. To simplify the playback logic for

the rolling sound, Marble Maze mutes the sound instead of stopping it. As the marble changes velocity, Marble Maze changes the pitch and volume of the sound to give it a more realistic effect. The following shows how the **MarbleMazeMain::Update** method updates the pitch and volume of the marble as its velocity changes and how it mutes the sound by setting its volume to zero when the marble stops.

C++

```
// Play the roll sound only if the marble is actually rolling.
if (ci.isRollingOnFloor && volume > 0)
{
    if (!m_audio.IsSoundEffectStarted(RollingEvent))
    {
        m_audio.PlaySoundEffect(RollingEvent);
    }

    // Update the volume and pitch by the velocity.
    m_audio.SetSoundEffectVolume(RollingEvent, volume);
    m_audio.SetSoundEffectPitch(RollingEvent, pitch);

    // The rolling sound has at most 8000Hz sounds, so we linearly
    // ramp up the low-pass filter the faster we go.
    // We also reduce the Q-value of the filter, starting with a
    // relatively broad cutoff and get progressively tighter.
    m_audio.SetSoundEffectFilter(
        RollingEvent,
        600.0f + 8000.0f * volume,
        XAUDIO2_MAX_FILTER_ONEOVERQ - volume*volume
        );
}
else
{
    m_audio.SetSoundEffectVolume(RollingEvent, 0);
}
```

Reacting to suspend and resume events

Marble Maze application structure describes how Marble Maze supports suspend and resume. When the game is suspended, the game pauses the audio. When the game resumes, the game resumes the audio where it left off. We do so to follow the best practice of not using resources when you know they're not needed.

The **Audio::SuspendAudio** method is called when the game is suspended. This method calls the IXAudio2::StopEngine method to stop all audio. Although **IXAudio2::StopEngine** stops all audio output immediately, it preserves the audio graph and its effect parameters (for example, the reverb effect that's applied when the marble bounces).

C++

```
// Uses the IXAudio2::StopEngine method to stop all audio immediately.
// It leaves the audio graph untouched, which preserves all effect
parameters
// and effect histories (like reverb effects) voice states, pending buffers,
// cursor positions and so on.
// When the engines are restarted, the resulting audio will sound as if it
had
// never been stopped except for the period of silence.
void Audio::SuspendAudio()
{
    if (m_engineExperiencedCriticalError)
    {
        return;
    }

    if (m_isAudioStarted)
    {
        m_musicEngine->StopEngine();
        m_soundEffectEngine->StopEngine();
    }

    m_isAudioStarted = false;
}
```

The **Audio::ResumeAudio** method is called when the game is resumed. This method uses the IXAudio2::StartEngine method to restart the audio. Because the call to IXAudio2::StopEngine preserves the audio graph and its effect parameters, the audio output resumes where it left off.

C++

```
// Restarts the audio streams. A call to this method must match a previous
call
// to SuspendAudio. This method causes audio to continue where it left off.
// If there is a problem with the restart, the
m_engineExperiencedCriticalError
// flag is set. The next call to Render will recreate all the resources and
// reset the audio pipeline.
void Audio::ResumeAudio()
{
    if (m_engineExperiencedCriticalError)
    {
        return;
    }

    HRESULT hr = m_musicEngine->StartEngine();
    HRESULT hr2 = m_soundEffectEngine->StartEngine();

    if (FAILED(hr) || FAILED(hr2))
    {
```

```
        m_engineExperiencedCriticalError = true;
    }
}
```

Handling headphones and device changes

Marble Maze uses engine callbacks to handle XAudio2 engine failures, such as when the audio device changes. A likely cause of a device change is when the game user connects or disconnects the headphones. We recommend that you implement the engine callback that handles device changes. Otherwise, your game will stop playing sound when the user plugs in or removes headphones, until the game is restarted.

Audio.h defines the **AudioEngineCallbacks** class. This class implements the IXAudio2EngineCallback interface.

C++

```
class AudioEngineCallbacks: public IXAudio2EngineCallback
{
private:
    Audio* m_audio;

public :
    AudioEngineCallbacks(){};
    void Initialize(Audio* audio);

    // Called by XAudio2 just before an audio processing pass begins.
    void _stdcall OnProcessingPassStart(){};

    // Called just after an audio processing pass ends.
    void  _stdcall OnProcessingPassEnd(){};

    // Called when a critical system error causes XAudio2
    // to be closed and restarted. The error code is given in Error.
    void  _stdcall OnCriticalError(HRESULT Error);
};
```

The IXAudio2EngineCallback interface enables your code to be notified when audio processing events occur and when the engine encounters a critical error. To register for callbacks, Marble Maze calls the IXAudio2::RegisterForCallbacks method in **Audio::CreateResources**, after it creates the IXAudio2 object for the music engine.

C++

```
m_musicEngineCallback.Initialize(this);
m_musicEngine->RegisterForCallbacks(&m_musicEngineCallback);
```

Marble Maze does not require notification when audio processing starts or ends. Therefore, it implements the IXAudio2EngineCallback::OnProcessingPassStart and IXAudio2EngineCallback::OnProcessingPassEnd methods to do nothing. For the IXAudio2EngineCallback::OnCriticalError method, Marble Maze calls the **SetEngineExperiencedCriticalError** method, which sets the **m_engineExperiencedCriticalError** flag.

C++

```
// Audio.cpp

// Called when a critical system error causes XAudio2
// to be closed and restarted. The error code is given in Error.
void  _stdcall AudioEngineCallbacks::OnCriticalError(HRESULT Error)
{
    m_audio->SetEngineExperiencedCriticalError();
}
```

C++

```
// Audio.h (Audio class)

// This flag can be used to tell when the audio system
// is experiencing critical errors.
// XAudio2 gives a critical error when the user unplugs
// the headphones and a new speaker configuration is generated.
void SetEngineExperiencedCriticalError()
{
    m_engineExperiencedCriticalError = true;
}
```

When a critical error occurs, audio processing stops and all additional calls to XAudio2 fail. To recover from this situation, you must release the XAudio2 instance and create a new one. The **Audio::Render** method, which is called from the game loop every frame, first checks the **m_engineExperiencedCriticalError** flag. If this flag is set, it clears the flag, releases the current XAudio2 instance, initializes resources, and then starts the background music.

C++

```
if (m_engineExperiencedCriticalError)
{
    m_engineExperiencedCriticalError = false;
    ReleaseResources();
    Initialize();
    CreateResources();
    Start();
    if (m_engineExperiencedCriticalError)
```

```
    {
        return;
    }
}
```

Marble Maze also uses the **m_engineExperiencedCriticalError** flag to guard against calling into XAudio2 when no audio device is available. For example, the **MarbleMazeMain::Update** method does not process audio for rolling or collision events when this flag is set. The app attempts to repair the audio engine every frame if it is required; however, the **m_engineExperiencedCriticalError** flag might always be set if the computer does not have an audio device or the headphones are unplugged and there is no other available audio device.

> **⊗ Caution**
>
> As a rule, do not perform blocking operations in the body of an engine callback. Doing so can cause performance issues. Marble Maze sets a flag in the **OnCriticalError** callback and later handles the error during the regular audio processing phase. For more information about XAudio2 callbacks, see **XAudio2 Callbacks**.

Conclusion

That wraps up the Marble Maze game sample! Though it is a relatively simple game, it contains many of the important parts that go into any UWP DirectX game, and is a good example to follow when making your own game.

Now that you've finished following along, try tinkering around with the source code and seeing what happens. Or check out Create a simple UWP game with DirectX, another UWP DirectX game sample.

Ready to go further with DirectX? Then check out our guides at DirectX programming.

If you're interested in game development on UWP in general, see the documentation at Game programming.

Related topics

- Adding input and interactivity to the Marble Maze sample
- Developing Marble Maze, a UWP game in C++ and DirectX

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Direct3D graphics glossary

Article • 10/20/2022

Defines Microsoft Direct3D graphics terms. This glossary defines, at a high level, general 3D computer graphics terms that are used in Direct3D game and app development.

In this section

Topic	Description
Coordinate systems and geometry	Programming Direct3D applications requires a working familiarity with 3D geometric principles. This section introduces the most important geometric concepts for creating 3D scenes.
Vertex and index buffers	*Vertex buffers* are memory buffers that contain vertex data; vertices in a vertex buffer are processed to perform transformation, lighting, and clipping. *Index buffers* are memory buffers that contain index data, which are integer offsets into vertex buffers, used to render primitives.
Devices	A Direct3D device is the rendering component of Direct3D. A device encapsulates and stores the rendering state, performs transformations and lighting operations, and rasterizes an image to a surface.
Lighting	Lights are used to illuminate objects in a scene. The color of each object vertex is based on the current texture map, vertex colors, and light sources.
Depth and stencil buffers	A *depth buffer* stores depth information to control which areas of polygons are rendered rather than hidden from view. A *stencil buffer* is used to mask pixels in an image, to produce special effects, including compositing; decaling; dissolves, fades, and swipes; outlines and silhouettes; and two-sided stencil.

Topic	Description
Textures	Textures are a powerful tool in creating realism in computer-generated 3D images. Direct3D supports an extensive texturing feature set, providing developers with easy access to advanced texturing techniques.
Graphics pipeline	The Direct3D graphics pipeline is designed for generating graphics for realtime gaming applications. Data flows from input to output through each of the configurable or programmable stages.
Views	The term "view" is used to mean "data in the required format". For example, a Constant Buffer View (CBV) would be constant buffer data correctly formatted. This section describes the most common and useful views.
Compute pipeline	The Direct3D compute pipeline is designed to handle calculations that can be done mostly in parallel with the graphics pipeline.
Resources	A resource is an area in memory that can be accessed by the Direct3D pipeline. In order for the pipeline to access memory efficiently, data that is provided to the pipeline (such as input geometry, shader resources, and textures) must be stored in a resource. There are two types of resources from which all Direct3D resources derive: a buffer or a texture. Up to 128 resources can be active for each pipeline stage.
Streaming resources	*Streaming resources* are large logical resources that use small amounts of physical memory. Instead of passing an entire large resource, small parts of the resource are streamed as needed. Streaming resources were previously called *tiled resources*.
Appendices	These sections provide in-depth technical details.

Coordinate systems and geometry

Article • 10/20/2022

Programming Direct3D applications requires a working familiarity with 3D geometric principles. This section introduces the most important geometric concepts for creating 3D scenes.

In this section

Topic	Description
Coordinate systems	Typically 3D graphics applications use one of two types of Cartesian coordinate systems: left-handed or right-handed. In both coordinate systems, the positive x-axis points to the right, and the positive y-axis points up.
Primitives	A 3D *primitive* is a collection of vertices that form a single 3D entity.
Face and vertex normal vectors	Each face in a mesh has a perpendicular unit normal vector. The vector's direction is determined by the order in which the vertices are defined and by whether the coordinate system is right- or left-handed.
Rectangles	Throughout Direct3D and Windows programming, objects on the screen are referred to in terms of bounding rectangles. The sides of a bounding rectangle are always parallel to the sides of the screen, so the rectangle can be described by two points, the upper-left corner and lower-right corner.
Triangle interpolation	During rendering, the pipeline interpolates vertex data across each triangle. Vertex data can be a broad variety of data and can include (but is not limited to): diffuse color, specular color, diffuse alpha (triangle opacity), specular alpha, and a fog factor.
Vectors, vertices, and quaternions	Throughout Direct3D, vertices describe position and orientation. Each vertex in a primitive is described by a vector that gives its position, color, texture coordinates, and a normal vector that gives its orientation.

Topic	Description
Transforms	The part of Direct3D that pushes geometry through the fixed function geometry pipeline is the transform engine. It locates the model and viewer in the world, projects vertices for display on the screen, and clips vertices to the viewport. The transform engine also performs lighting computations to determine diffuse and specular components at each vertex.
Viewports and clipping	A *viewport* is a two-dimensional (2D) rectangle into which a 3D scene is projected. In Direct3D, the rectangle exists as coordinates within a Direct3D surface that the system uses as a rendering target. The projection transformation converts vertices into the coordinate system used for the viewport. A viewport is also used to specify the range of depth values on a render-target surface into which a scene will be rendered (usually 0.0 to 1.0).

Related topics

Direct3D Graphics Learning Guide

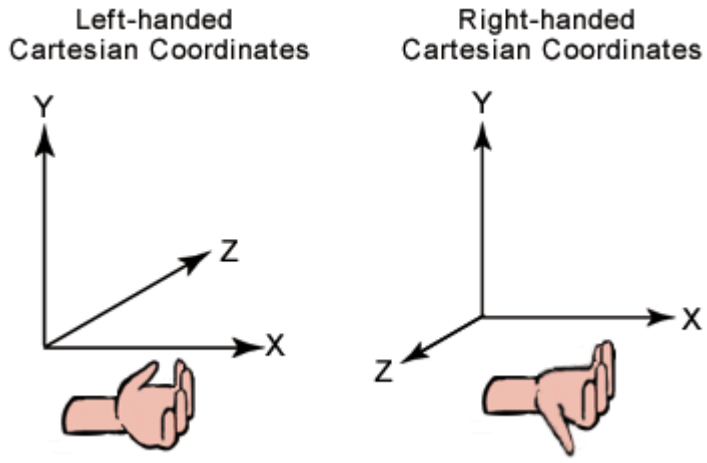
Coordinate systems

Article • 10/20/2022

Typically 3D graphics applications use one of two types of Cartesian coordinate systems: left-handed or right-handed. In both coordinate systems, the positive x-axis points to the right, and the positive y-axis points up.

Left and right handed coordinates

You can remember which direction the positive z-axis points by pointing the fingers of either your left or right hand in the positive x-direction and curling them into the positive y-direction. The direction your thumb points, either toward or away from you, is the direction that the positive z-axis points for that coordinate system. The following illustration shows these two coordinate systems.



Direct3D uses a left-handed coordinate system. Although left-handed and right-handed coordinates are the most common systems, there is a variety of other coordinate systems used in 3D software. For example, it is not unusual for 3D modeling applications to use a coordinate system in which the y-axis points toward or away from the viewer, and the z-axis points up.

Vertices and vectors

Given the coordinate system, an x, y and z coordinate can define a point in space (a "vertex"), or a 3D direction (a "vector").

A collection of vertices can be used to define lines and shapes. The simplest objects definable by vertices are called Primitives, and a more complex object defined by a set of primitives is called a "mesh."

The essential operations performed on meshes defined in a 3D coordinate system are translation, rotation, and scaling. You can combine these basic transformations to create a transform matrix. For details, see Transforms.

When you combine these operations, the results are not commutative; the order in which you multiply matrices is important.

Porting from a right-handed coordinate system

If you are porting an application that is based on a right-handed coordinate system, you must make two changes to the data passed to Direct3D:

- Flip the order of triangle vertices so that the system traverses them clockwise from the front. In other words, if the vertices are v0, v1, v2, pass them to Direct3D as v0, v2, v1.
- Use the view matrix to scale world space by -1 in the z-direction. To do this, flip the sign of the _31, _32, _33, and _34 member of the matrix structure that you use for your view matrix.

Related topics

Coordinate systems and geometry

Primitives

Article • 10/20/2022

A 3D *primitive* is a collection of vertices that form a single 3D entity.

In this section

Topic	Description
Point lists	A point list is a collection of vertices that are rendered as isolated points. Your application can use point lists in 3D scenes for star fields, or dotted lines on the surface of a polygon.
Line lists	A line list is a list of isolated, straight line segments. Line lists are useful for such tasks as adding sleet or heavy rain to a 3D scene. Applications create a line list by filling an array of vertices.
Line strips	A line strip is a primitive that is composed of connected line segments. Your application can use line strips for creating polygons that are not closed. A closed polygon is a polygon whose last vertex is connected to its first vertex by a line segment.
Triangle lists	A triangle list is a list of isolated triangles. The isolated triangles might or might not be near each other. A triangle list must have at least three vertices and the total number of vertices must be divisible by three.
Triangle strips	A triangle strip is a series of connected triangles. Because the triangles are connected, the application does not need to repeatedly specify all three vertices for each triangle.

Related topics

Coordinate systems and geometry

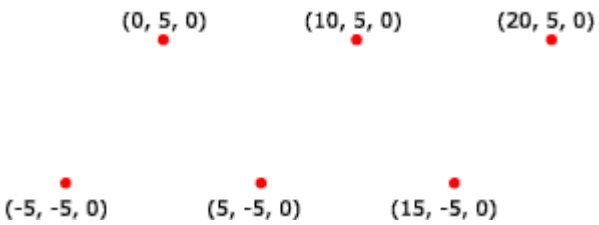
Point lists

Article • 10/20/2022

A point list is a collection of vertices that are rendered as isolated points. Your application can use point lists in 3D scenes for star fields, or dotted lines on the surface of a polygon.

Example

The following illustration depicts a rendered point list.



Your application can apply materials and textures to a point list. The colors in the material or texture appear only at the points drawn, and not anywhere between the points.

The following code shows how to create vertices for this point list.

C++

```
struct CUSTOMVERTEX
{
    float x,y,z;
};

CUSTOMVERTEX Vertices[] =
{
    {-5.0, -5.0, 0.0},
    { 0.0,  5.0, 0.0},
    { 5.0, -5.0, 0.0},
    {10.0,  5.0, 0.0},
    {15.0, -5.0, 0.0},
    {20.0,  5.0, 0.0}
};
```

The code example below shows how to render this point list in Direct3D.

C++

```
//
// It is assumed that d3dDevice is a valid
```

```
// pointer to an IDirect3DDevice interface.
//
d3dDevice->DrawPrimitive( D3DPT_POINTLIST, 0, 6 );
```

Related topics

Primitives

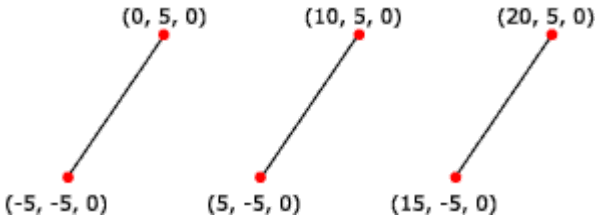
Line lists

Article • 10/20/2022

A line list is a list of isolated, straight line segments. Line lists are useful for such tasks as adding sleet or heavy rain to a 3D scene. Applications create a line list by filling an array of vertices. Note that the number of vertices in a line list must be an even number greater than or equal to two.

Example

The following illustration shows a rendered line list.



You can apply materials and textures to a line list. The colors in the material or texture appear only along the lines drawn, not at any point in between the lines.

The following code shows how to create vertices for this line list.

C++

```
struct CUSTOMVERTEX
{
    float x,y,z;
};

CUSTOMVERTEX Vertices[] =
{
    {-5.0, -5.0, 0.0},
    { 0.0,  5.0, 0.0},
    { 5.0, -5.0, 0.0},
    {10.0,  5.0, 0.0},
    {15.0, -5.0, 0.0},
    {20.0,  5.0, 0.0}
};
```

The code example below shows how to render a line list in Direct3D.

C++

```
//
// It is assumed that d3dDevice is a valid
// pointer to an IDirect3DDevice interface.
//
d3dDevice->DrawPrimitive( D3DPT_LINELIST, 0, 3 );
```

Related topics

Primitives

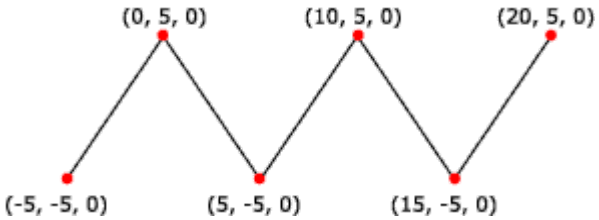
Line strips

Article • 10/20/2022

A line strip is a primitive that is composed of connected line segments. Your application can use line strips for creating polygons that are not closed. A closed polygon is a polygon whose last vertex is connected to its first vertex by a line segment. If your application makes polygons based on line strips, the vertices are not guaranteed to be coplanar.

Example

The following illustration shows a rendered line strip.



The following code shows how to create vertices for this line strip.

C++

```
struct CUSTOMVERTEX
{
    float x,y,z;
};

CUSTOMVERTEX Vertices[] =
{
    {-5.0, -5.0, 0.0},
    { 0.0,  5.0, 0.0},
    { 5.0, -5.0, 0.0},
    {10.0,  5.0, 0.0},
    {15.0, -5.0, 0.0},
    {20.0,  5.0, 0.0}
};
```

The code example below shows how to render a line strip in Direct3D.

C++

```
//
// It is assumed that d3dDevice is a valid
// pointer to an IDirect3DDevice interface.
```

```
//
d3dDevice->DrawPrimitive( D3DPT_LINESTRIP, 0, 5 );
```

Related topics

Primitives

Triangle lists

Article • 10/20/2022

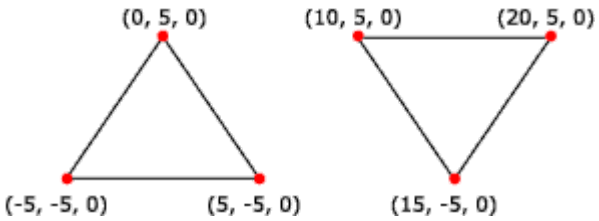
A triangle list is a list of isolated triangles. The isolated triangles might or might not be near each other. A triangle list must have at least three vertices and the total number of vertices must be divisible by three.

Example

Use triangle lists to create an object that is composed of disjoint pieces. For instance, one way to create a force-field wall in a 3D game is to specify a large list of small, unconnected triangles. Then apply a material and texture that appears to emit light to the triangle list. Each triangle in the wall appears to glow. The scene behind the wall becomes partially visible through the gaps between the triangles, as a player might expect when looking at a force field.

Triangle lists are also useful for creating primitives that have sharp edges and are shaded with Gouraud shading. See Face and vertex normal vectors.

The following illustration depicts a rendered triangle list.



The following code shows how to create vertices for this triangle list.

C++

```
struct CUSTOMVERTEX
{
    float x,y,z;
};

CUSTOMVERTEX Vertices[] =
{
    {-5.0, -5.0, 0.0},
    { 0.0,  5.0, 0.0},
    { 5.0, -5.0, 0.0},
    {10.0,  5.0, 0.0},
    {15.0, -5.0, 0.0},
    {20.0,  5.0, 0.0}

};
```

The code example below shows how to render this triangle list in Direct3D.

```
C++
```

```
//
// It is assumed that d3dDevice is a valid
// pointer to a device interface.
//
d3dDevice->DrawPrimitive( D3DPT_TRIANGLELIST, 0, 2 );
```

Related topics

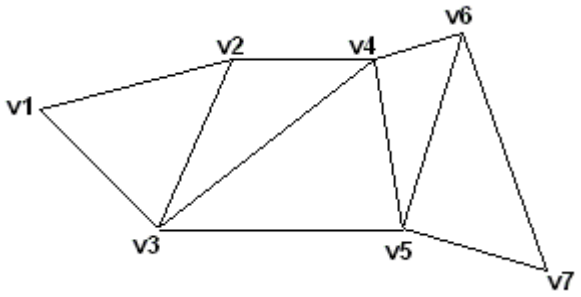
Primitives

Triangle strips

Article • 10/20/2022

A triangle strip is a series of connected triangles. Because the triangles are connected, the application does not need to repeatedly specify all three vertices for each triangle. For example, you need only seven vertices to define the following triangle strip.

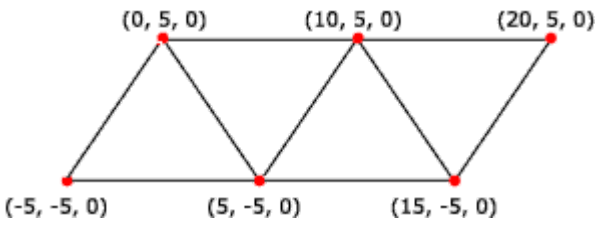
Example



The system uses vertices v1, v2, and v3 to draw the first triangle; v2, v4, and v3 to draw the second triangle; v3, v4, and v5 to draw the third; v4, v6, and v5 to draw the fourth; and so on. Notice that the vertices of the second and fourth triangles are out of order; this is required to make sure that all the triangles are drawn in a clockwise orientation.

Most objects in 3D scenes are composed of triangle strips. This is because triangle strips can be used to specify complex objects in a way that makes efficient use of memory and processing time.

The following illustration depicts a rendered triangle strip.



The following code shows how to create vertices for this triangle strip.

C++

```
struct CUSTOMVERTEX
{
float x,y,z;
};

CUSTOMVERTEX Vertices[] =
{
```

```
    {-5.0, -5.0, 0.0},
    { 0.0,  5.0, 0.0},
    { 5.0, -5.0, 0.0},
    {10.0,  5.0, 0.0},
    {15.0, -5.0, 0.0},
    {20.0,  5.0, 0.0}
};
```

The code example below shows how to render this triangle strip in Direct3D.

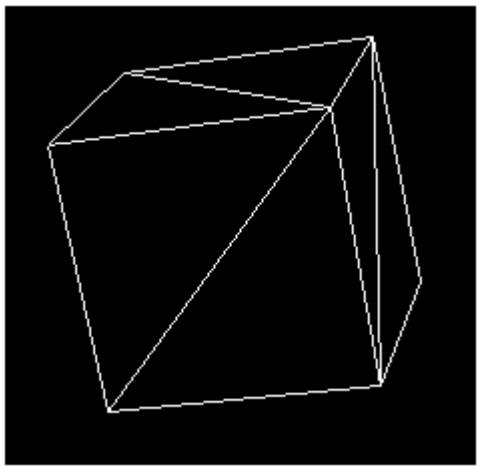
C++

```
//
// It is assumed that d3dDevice is a valid
// pointer to a device interface.
//
d3dDevice->DrawPrimitive( D3DPT_TRIANGLESTRIP, 0, 4);
```

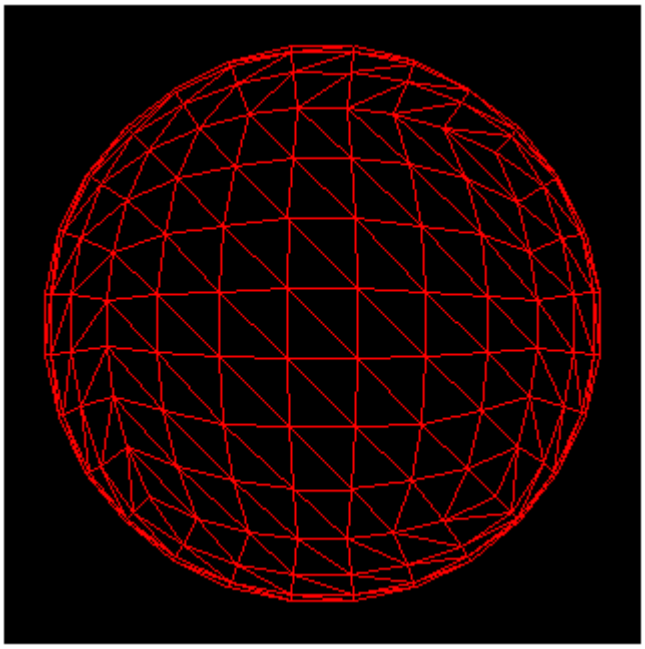
Polygons

Often, triangle strips are used to build polygons. A polygon is a closed 3D figure delineated by at least three vertices. The simplest polygon is a triangle. Microsoft Direct3D uses triangles to compose most of its polygons because all three vertices in a triangle are guaranteed to be coplanar. Rendering nonplanar vertices is inefficient. You can combine triangles to form large, complex polygons and meshes.

The following illustration shows a cube. Two triangles form each face of the cube. The entire set of triangles forms one cubic primitive. You can apply textures to the surfaces of primitives to make them appear to be a single solid form. For details, see Textures.



You can also use triangles to create primitives whose surfaces appear to be smooth curves. The following illustration shows how a sphere can be simulated with triangles. After a material is applied, the sphere can be made to look curved when it is rendered.



Related topics

Primitives

Face and vertex normal vectors

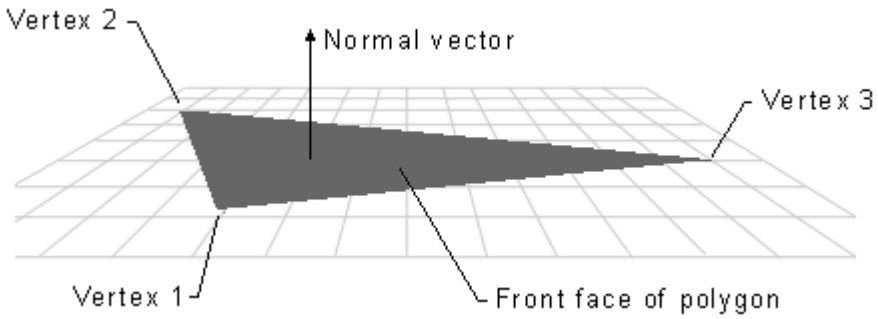
Article • 10/20/2022

Each face in a mesh has a perpendicular unit normal vector. The vector's direction is determined by the order in which the vertices are defined and by whether the coordinate system is right- or left-handed.

Perpendicular unit normal vector for a front face

Each face in a mesh has a perpendicular unit normal vector. The vector's direction is determined by the order in which the vertices are defined and by whether the coordinate system is right- or left-handed. The face normal points away from the front side of the face. In Direct3D, only the front of a face is visible. A front face is one in which vertices are defined in clockwise order.

The following illustration shows a normal vector for a front face:



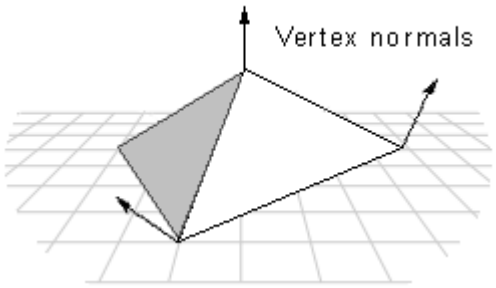
Culling back faces

Any face that is not a front face is a back face. Direct3D does not always render back faces; back faces are said to be culled. Back face culling means eliminating back faces from rendering. You can change the culling mode to render back faces if you want. See Culling State for more information.

Vertex unit normals

Direct3D uses the vertex unit normals for Gouraud shading, lighting, and texturing effects.

The following illustration shows vertex normals:

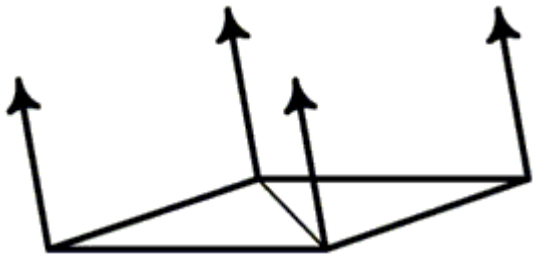


When applying Gouraud shading to a polygon, Direct3D uses the vertex normals to calculate the angle between the light source and the surface. It calculates the color and intensity values for the vertices and interpolates them for every point across all the primitive's surfaces. Direct3D calculates the light intensity value by using the angle. The greater the angle, the less light is shining on the surface.

Flat surfaces

If you are creating an object that is flat, set the vertex normals to point perpendicular to the surface.

The following illustration shows a flat surface composed of two triangles with vertex normals:



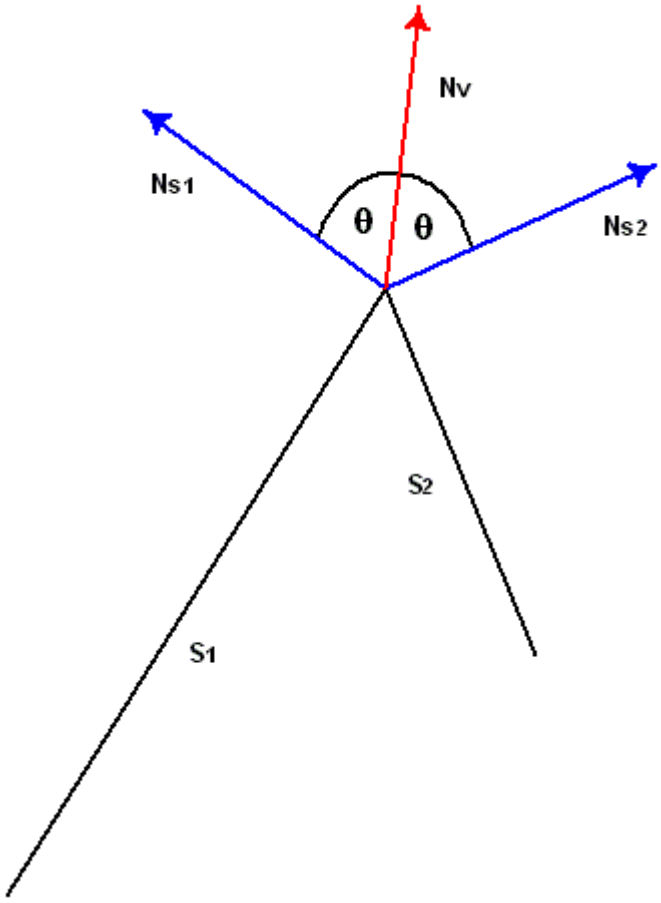
Smooth shading on a non-flat object

Rather than flat object, it is more likely that your object is made up of triangle strips and the triangles are not coplanar. One simple way to achieve smooth shading across all the triangles in the strip is to first calculate the surface normal vector for each polygonal face with which the vertex is associated. The vertex normal can be set to make an equal angle with each surface normal. However, this method might not be efficient enough for complex primitives.

This method is illustrated by the following diagram, which shows two surfaces, S1 and S2 seen edge-on from above. The normal vectors for S1 and S2 are shown in blue. The vertex normal vector is shown in red. The angle that the vertex normal vector makes with the surface normal of S1 is the same as the angle between the vertex normal and

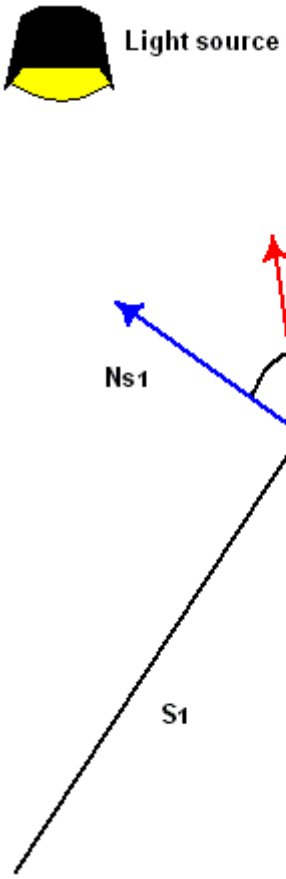
the surface normal of S2. When these two surfaces are lit and shaded with Gouraud shading, the result is a smoothly shaded, smoothly rounded edge between them.

The following illustration shows two surfaces (S1 and S2) and their normal vectors and vertex normal vector:



If the vertex normal leans toward one of the faces with which it is associated, it causes the light intensity to increase or decrease for points on that surface, depending on the angle it makes with the light source. The following diagram shows an example. These surfaces are seen edge-on. The vertex normal leans toward S1, causing it to have a smaller angle with the light source than if the vertex normal had equal angles with the surface normals.

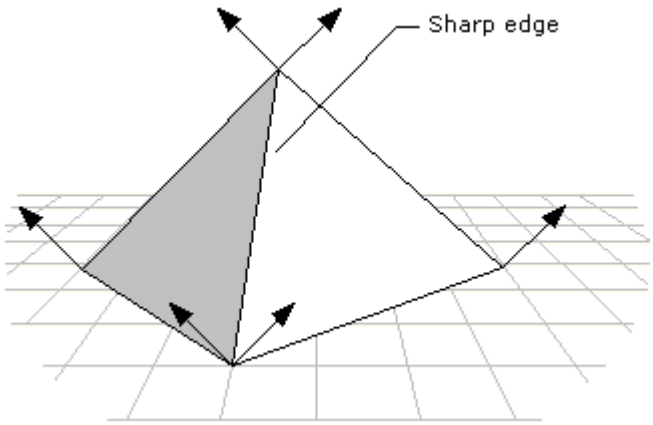
The following illustration shows two surfaces (S1 and S2) with a vertex normal vector that leans toward one face:



Sharp edges

You can use Gouraud shading to display some objects in a 3D scene with sharp edges. To do so, duplicate the vertex normal vectors at any intersection of faces where a sharp edge is required.

The following illustration shows duplicated vertex normal vectors at sharp edges:



If you use the DrawPrimitive methods to render your scene, define the object with sharp edges as a triangle list, rather than a triangle strip. When you define an object as a triangle strip, Direct3D treats it as a single polygon composed of multiple triangular faces. Gouraud shading is applied both across each face of the polygon and between adjacent faces.

The result is an object that is smoothly shaded from face to face. Because a triangle list is a polygon composed of a series of disjoint triangular faces, Direct3D applies Gouraud shading across each face of the polygon. However, it is not applied from face to face. If two or more triangles of a triangle list are adjacent, they appear to have a sharp edge between them.

Another alternative is to change to flat shading when rendering objects with sharp edges. This is computationally the most efficient method, but it may result in objects in the scene that are not rendered as realistically as the objects that are Gouraud-shaded.

Related topics

Coordinate systems and geometry

Rectangles

Article • 11/29/2022

Throughout Direct3D and Windows programming, objects on the screen are referred to in terms of bounding rectangles. The sides of a bounding rectangle are always parallel to the sides of the screen, so the rectangle can be described by two points, the upper-left corner and lower-right corner.

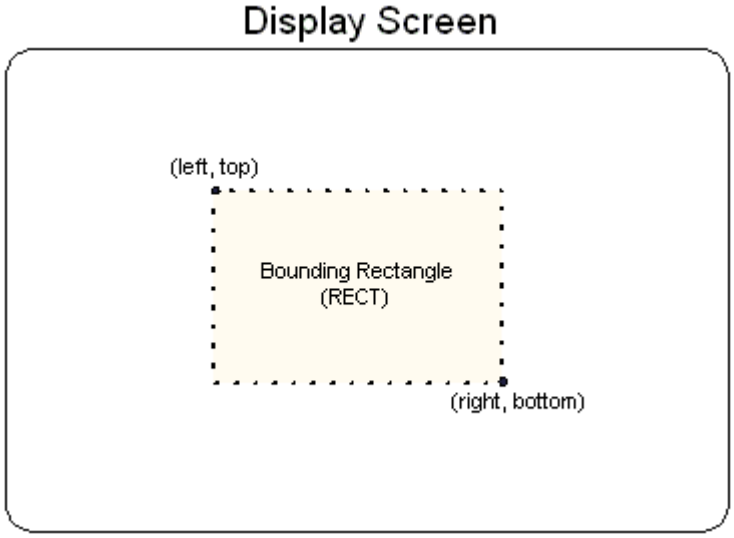
Bounding rectangles

Most applications use the **RECT** structure (or a typedef'd alias for it) to carry information about a bounding rectangle to use when blitting to the screen or when performing hit detection. In C++, the **RECT** structure has the following definition.

C++

```
typedef struct tagRECT {
    LONG    left;    // This is the upper-left corner x-coordinate.
    LONG    top;     // The upper-left corner y-coordinate.
    LONG    right;   // The lower-right corner x-coordinate.
    LONG    bottom;  // The lower-right corner y-coordinate.
} RECT, *PRECT, NEAR *NPRECT, FAR *LPRECT;
```

In the preceding example, the left and top members are the x- and y-coordinates of a bounding rectangle's upper-left corner. Similarly, the right and bottom members make up the coordinates of the lower-right corner. The following illustration shows how you can visualize these values.



The column of pixels at the right edge and the row of pixels at the bottom edge are not included in the RECT. For example, locking a RECT with members {10, 10, 138, 138}

results in an object 128 pixels in width and height.

For efficiency, consistency, and ease of use, all Direct3D presentation functions work with rectangles.

Related topics

Coordinate systems and geometry

Triangle interpolation

Article • 10/20/2022

During rendering, the pipeline interpolates vertex data across each triangle. Vertex data can be a broad variety of data and can include (but is not limited to): diffuse color, specular color, diffuse alpha (triangle opacity), specular alpha, and a fog factor. For the programmable vertex pipeline, the fog factor is taken from the fog register. For the fixed-function vertex pipeline, the fog factor is taken from specular alpha.

For some vertex data, the interpolation is dependent on the current shading mode, as follows:

Shading mode	Description
Flat	Only the fog factor is interpolated in flat shade mode. For all other interpolated values, the color of the first vertex in the triangle is applied across the entire face.
Gouraud	Linear interpolation is performed between all three vertices.

The diffuse color and specular color are treated differently, depending on the color model. In the RGB color model, the system uses the red, green, and blue color components in the interpolation.

The alpha component of a color is treated as a separate interpolated value because device drivers can implement transparency in two different ways: by using texture blending or by using stippling.

Related topics

Coordinate systems and geometry

Vectors, vertices, and quaternions

Article • 10/20/2022

Throughout Direct3D, vertices describe position and orientation. Each vertex in a primitive is described by a vector that gives its position, color, texture coordinates, and a normal vector that gives its orientation.

Quaternions add a fourth element to the [x, y, z] values that define a three-component-vector. Quaternions are an alternative to the matrix methods that are typically used for 3D rotations. A quaternion represents an axis in 3D space and a rotation around that axis. For example, a quaternion might represent a (1,1,2) axis and a rotation of 1 radian. Quaternions carry valuable information, but their true power comes from the two operations that you can perform on them: composition and interpolation.

Performing composition on quaternions is similar to combining them. The composition of two quaternions is notated like the following illustration.

$$Q = q_1 \circ q_2$$

The composition of two quaternions applied to a geometry means "rotate the geometry around $axis_2$ by $rotation_2$, then rotate it around $axis_1$ by $rotation_1$." In this case, Q represents a rotation around a single axis that is the result of applying q_2, then q_1 to the geometry.

Using quaternion interpolation, an application can calculate a smooth and reasonable path from one axis and orientation to another. Therefore, interpolation between q_1 and q_2 provides a simple way to animate from one orientation to another.

When you use composition and interpolation together, they provide you with a simple way to manipulate a geometry in a manner that appears complex. For example, imagine that you have a geometry that you want to rotate to a given orientation. You know that you want to rotate it r_2 degrees around $axis_2$, then rotate it r_1 degrees around $axis_1$, but you don't know the final quaternion. By using composition, you could combine the two rotations on the geometry to get a single quaternion that is the result. Then, you could interpolate from the original to the composed quaternion to achieve a smooth transition from one to the other.

Related topics

Coordinate systems and geometry

Transforms

Article • 10/20/2022

The part of Direct3D that pushes geometry through the fixed function geometry pipeline is the transform engine. It locates the model and viewer in the world, projects vertices for display on the screen, and clips vertices to the viewport. The transform engine also performs lighting computations to determine diffuse and specular components at each vertex.

In this section

Topic	Description
Transform overview	Matrix transformations handle a lot of the low level math of 3D graphics.
World transform	A world transform changes coordinates from model space, where vertices are defined relative to a model's local origin, to world space. In world space, vertices are defined relative to an origin common to all the objects in a scene. The world transform places a model into the world.
View transform	A *view transform* locates the viewer in world space, transforming vertices into camera space. In camera space, the camera, or viewer, is at the origin, looking in the positive z-direction. The view matrix relocates the objects in the world around a camera's position - the origin of camera space - and orientation.
Projection transform	A *projection transformation* controls the camera's internals, like choosing a lens for a camera. This is the most complicated of the three transformation types.

Related topics

Coordinate systems and geometry

Transform overview

Article • 10/20/2022

Matrix transformations handle a lot of the low level math of 3D graphics.

The geometry pipeline takes vertices as input. The transform engine applies the world, view, and projection transforms to the vertices, clips the result, and passes everything to the rasterizer.

Transform and space	Description
Model coordinates in model space	At the head of the pipeline, a model's vertices are declared relative to a local coordinate system. This is a local origin and an orientation. This orientation of coordinates is often referred to as *model space*. Individual coordinates are called *model coordinates*.
World transform into world space	The first stage of the geometry pipeline transforms a model's vertices from their local coordinate system to a coordinate system that is used by all the objects in a scene. The process of reorienting the vertices is called the World transform, which converts from model space to a new orientation called *world space*. Each vertex in world space is declared using *world coordinates*.
View transform into view space (camera space)	In the next stage, the vertices that describe your 3D world are oriented with respect to a camera. That is, your application chooses a point-of-view for the scene, and world space coordinates are relocated and rotated around the camera's view, turning world space into *view space* (also known as *camera space*). This is the View transform, which converts from world space to view space.
Projection transform into projection space	The next stage is the Projection transform, which converts from view space to projection space. In this part of the pipeline, objects are usually scaled with relation to their distance from the viewer in order to give the illusion of depth to a scene; close objects are made to appear larger than distant objects. For simplicity, this documentation refers to the space in which vertices exist after the projection transform as *projection space*. Some graphics books might refer to projection space as *post-perspective homogeneous space*. Not all projection transforms scale the size of objects in a scene. A projection such as this is sometimes called an *affine* or *orthogonal projection*.
Clipping in screen space	In the final part of the pipeline, any vertices that will not be visible on the screen are removed, so that the rasterizer doesn't take the time to calculate the colors and shading for something that will never be seen. This process is called *clipping*. After clipping, the remaining vertices are scaled according to the viewport parameters and converted into screen coordinates. The resulting vertices, seen on the screen when the scene is rasterized, exist in *screen space*.

Transforms are used to convert object geometry from one coordinate space to another. Direct3D uses matrices to perform 3D transforms. Matrices create 3D transforms. You can combine matrices to produce a single matrix that encompasses multiple transforms.

You can transform coordinates between model space, world space, and view space.

- World transform - Converts from model space to world space.
- View transform - Converts from world space to view space.
- Projection transform - Converts from view space to projection space.

Matrix Transforms

In applications that work with 3D graphics, you can use geometrical transforms to do the following:

- Express the location of an object relative to another object.
- Rotate and size objects.
- Change viewing positions, directions, and perspectives.

You can transform any point (x,y,z) into another point (x', y', z') by using a 4x4 matrix, as shown in the following equation.

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} M_{11} & M_{12} & M_{13} & M_{14} \\ M_{21} & M_{22} & M_{23} & M_{24} \\ M_{31} & M_{32} & M_{33} & M_{34} \\ M_{41} & M_{42} & M_{43} & M_{44} \end{bmatrix}$$

Perform the following equations on (x, y, z) and the matrix to produce the point (x', y', z').

$$x' = (x \times M_{11}) + (y \times M_{21}) + (z \times M_{31}) + (1 \times M_{41})$$
$$y' = (x \times M_{12}) + (y \times M_{22}) + (z \times M_{32}) + (1 \times M_{42})$$
$$z' = (x \times M_{13}) + (y \times M_{23}) + (z \times M_{33}) + (1 \times M_{43})$$

The most common transforms are translation, rotation, and scaling. You can combine the matrices that produce these effects into a single matrix to calculate several transforms at once. For example, you can build a single matrix to translate and rotate a series of points.

Matrices are written in row-column order. A matrix that evenly scales vertices along each axis, known as uniform scaling, is represented by the following matrix using

mathematical notation.

$$\begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In C++, Direct3D declares matrices as a two-dimensional array, using a matrix struct. The following example shows how to initialize a **D3DMATRIX** structure to act as a uniform scaling matrix (scale factor "s").

C++

```
D3DMATRIX scale = {
    5.0f,           0.0f,           0.0f,           0.0f,
    0.0f,           5.0f,           0.0f,           0.0f,
    0.0f,           0.0f,           5.0f,           0.0f,
    0.0f,           0.0f,           0.0f,           1.0f
};
```

Translate

The following equation translates the point (x, y, z) to a new point (x', y', z').

$$[x'y'z'1] = [x\ y\ z\ 1]\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ T_x & T_y & T_z & 1 \end{bmatrix}$$

You can manually create a translation matrix in C++. The following example shows the source code for a function that creates a matrix to translate vertices.

C++

```
D3DXMATRIX Translate(const float dx, const float dy, const float dz) {
    D3DXMATRIX ret;

    D3DXMatrixIdentity(&ret);
    ret(3, 0) = dx;
    ret(3, 1) = dy;
    ret(3, 2) = dz;
    return ret;
}    // End of Translate
```

Scale

The following equation scales the point (x, y, z) by arbitrary values in the x-, y-, and z-directions to a new point (x', y', z').

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotate

The transforms described here are for left-handed coordinate systems, and so may be different from transform matrices that you have seen elsewhere.

The following equation rotates the point (x, y, z) around the x-axis, producing a new point (x', y', z').

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & \sin\theta & 0 \\ 0 & -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The following equation rotates the point around the y-axis.

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & 0 & -\sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The following equation rotates the point around the z-axis.

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 & 0 \\ -\sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In these example matrices, the Greek letter theta stands for the angle of rotation, in radians. Angles are measured clockwise when looking along the rotation axis toward the origin.

The following code shows a function to handle rotation about the X axis.

```
C++
```

```
    // Inputs are a pointer to a matrix (pOut) and an angle in radians.
    float sin, cos;
    sincosf(angle, &sin, &cos);  // Determine sin and cos of angle

    pOut->_11 = 1.0f; pOut->_12 =  0.0f;   pOut->_13 = 0.0f; pOut->_14 =
0.0f;
    pOut->_21 = 0.0f; pOut->_22 =  cos;    pOut->_23 = sin;  pOut->_24 =
0.0f;
    pOut->_31 = 0.0f; pOut->_32 = -sin;    pOut->_33 = cos;  pOut->_34 =
0.0f;
    pOut->_41 = 0.0f; pOut->_42 =  0.0f;   pOut->_43 = 0.0f; pOut->_44 =
1.0f;

    return pOut;
}
```

Concatenating Matrices

One advantage of using matrices is that you can combine the effects of two or more matrices by multiplying them. This means that, to rotate a model and then translate it to some location, you don't need to apply two matrices. Instead, you multiply the rotation and translation matrices to produce a composite matrix that contains all their effects. This process, called matrix concatenation, can be written with the following equation.

$$C = M_1 \cdot M_2 \cdot M_{n-1} \cdot M_n$$

In this equation, C is the composite matrix being created, and M_1 through M_n are the individual matrices. In most cases, only two or three matrices are concatenated, but there is no limit.

The order in which the matrix multiplication is performed is crucial. The preceding formula reflects the left-to-right rule of matrix concatenation. That is, the visible effects of the matrices that you use to create a composite matrix occur in left-to-right order. A typical world matrix is shown in the following example. Imagine that you are creating the world matrix for a stereotypical flying saucer. You would probably want to spin the flying saucer around its center - the y-axis of model space - and translate it to some other location in your scene. To accomplish this effect, you first create a rotation matrix, and then multiply it by a translation matrix, as shown in the following equation.

$$W = R_y \cdot T_w$$

In this formula, R_y is a matrix for rotation about the y-axis, and T_w is a translation to some position in world coordinates.

The order in which you multiply the matrices is important because, unlike multiplying two scalar values, matrix multiplication is not commutative. Multiplying the matrices in the opposite order has the visual effect of translating the flying saucer to its world space position, and then rotating it around the world origin.

No matter what type of matrix you are creating, remember the left-to-right rule to ensure that you achieve the expected effects.

Related topics

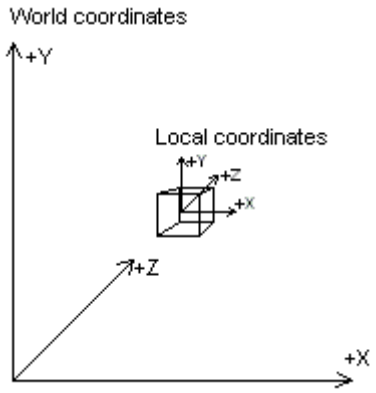
Transforms

World transform

Article • 10/20/2022

A world transform changes coordinates from model space, where vertices are defined relative to a model's local origin, to world space. In world space, vertices are defined relative to an origin common to all the objects in a scene. The world transform places a model into the world.

The following diagram shows the relationship between the world coordinate system and a model's local coordinate system.



The world transform can include any combination of translations, rotations, and scalings.

Setting Up a World Matrix

As with any other transform, create the world transform by concatenating a series of matrices into a single matrix that contains the sum total of their effects. In the simplest case, when a model is at the world origin and its local coordinate axes are oriented the same as world space, the world matrix is the identity matrix. More commonly, the world matrix is a combination of a translation into world space and possibly one or more rotations to turn the model as needed.

Direct3D uses the world and view matrices that you set to configure several internal data structures. Each time you set a new world or view matrix, the system recalculates the associated internal structures. Setting these matrices frequently-for example, thousands of times per frame-is computationally time-consuming. You can minimize the number of required calculations by concatenating your world and view matrices into a world-view matrix that you set as the world matrix, and then setting the view matrix to the identity. Keep cached copies of individual world and view matrices so that you can modify, concatenate, and reset the world matrix as needed.

Related topics

Transforms

View transform

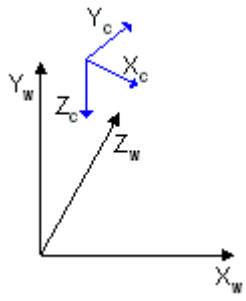
Article • 10/20/2022

A *view transform* locates the viewer in world space, transforming vertices into camera space. In camera space, the camera, or viewer, is at the origin, looking in the positive z-direction. The view matrix relocates the objects in the world around a camera's position - the origin of camera space - and orientation. Direct3D uses a left-handed coordinate system, so z is positive into a scene.

There are many ways to create a view matrix. In all cases, the camera has some logical position and orientation in world space that is used as a starting point to create a view matrix that will be applied to the models in a scene. The view matrix translates and rotates objects to place them in camera space, where the camera is at the origin. One way to create a view matrix is to combine a translation matrix with rotation matrices for each axis. In this approach, the following general matrix equation applies.

$$V = T \cdot R_z \cdot R_y \cdot R_x$$

In this formula, V is the view matrix being created, T is a translation matrix that repositions objects in the world, and R_x through R_z are rotation matrices that rotate objects along the x-, y-, and z-axis. The translation and rotation matrices are based on the camera's logical position and orientation in world space. So, if the camera's logical position in the world is <10,20,100>, the aim of the translation matrix is to move objects -10 units along the x-axis, -20 units along the y-axis, and -100 units along the z-axis. The rotation matrices in the formula are based on the camera's orientation, in terms of how much the axes of camera space are rotated out of alignment with world space. For example, if the camera mentioned earlier is pointing straight down, its z-axis is 90 degrees (pi/2 radians) out of alignment with the z-axis of world space, as shown in the following illustration.



The rotation matrices apply rotations of equal, but opposite, magnitude to the models in the scene. The view matrix for this camera includes a rotation of -90 degrees around the x-axis. The rotation matrix is combined with the translation matrix to create a view

matrix that adjusts the position and orientation of the objects in the scene so that their top is facing the camera, giving the appearance that the camera is above the model.

Setting Up a View Matrix

Direct3D uses the world and view matrices to configure several internal data structures. Each time you set a new world or view matrix, the system recalculates the associated internal structures. Setting these matrices frequently is computationally time-consuming. You can minimize the number of required calculations by concatenating your world and view matrices into a world-view matrix that you set as the world matrix, and then setting the view matrix to the identity. Keep cached copies of individual world and view matrices that you can modify, concatenate, and reset the world matrix as needed.

Related topics

Transforms

Projection transform

Article • 10/20/2022

A *projection transformation* controls the camera's internals, like choosing a lens for a camera. This is the most complicated of the three transformation types.

The projection matrix is typically a scale and perspective projection. The projection transformation converts the viewing frustum into a cuboid shape. Because the near end of the viewing frustum is smaller than the far end, this has the effect of expanding objects that are near to the camera; this is how perspective is applied to the scene.

In the viewing frustum, the distance between the camera and the origin of the viewing transform space is defined arbitrarily as D, so the projection matrix looks like the following illustration.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1/D \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

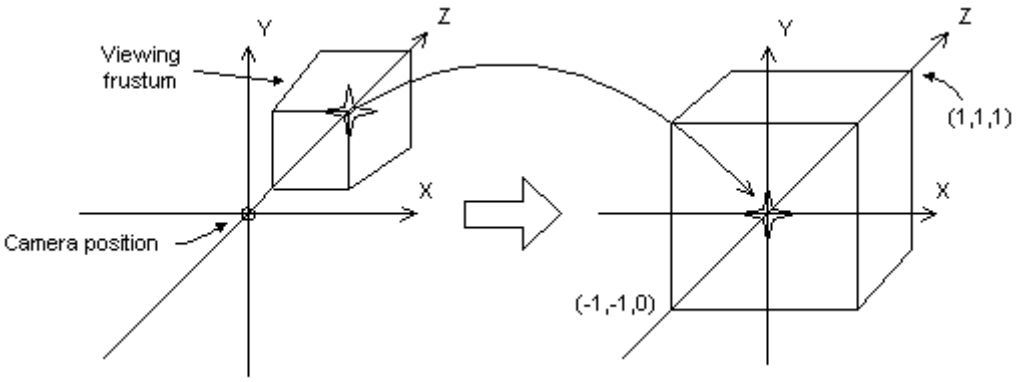
The viewing matrix translates the camera to the origin by translating in the z direction by - D. The translation matrix is like the following illustration.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -D & 1 \end{bmatrix}$$

Multiplying the translation matrix by the projection matrix (T*P) gives the composite projection matrix, as shown in the following illustration.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1/D \\ 0 & 0 & -D & 0 \end{bmatrix}$$

The perspective transform converts a viewing frustum into a new coordinate space. Notice that the frustum becomes cuboid and also that the origin moves from the upper-right corner of the scene to the center, as shown in the following diagram.



In the perspective transform, the limits of the x- and y-directions are -1 and 1. The limits of the z-direction are 0 for the front plane and 1 for the back plane.

This matrix translates and scales objects based on a specified distance from the camera to the near clipping plane, but it doesn't consider the field of view (fov), and the z-values that it produces for objects in the distance can be nearly identical, making depth comparisons difficult. The following matrix addresses these issues, and it adjusts vertices to account for the aspect ratio of the viewport, making it a good choice for the perspective projection.

$$\begin{bmatrix} w & 0 & 0 & 0 \\ 0 & h & 0 & 0 \\ 0 & 0 & Q & 1 \\ 0 & 0 & -QZ_n & 0 \end{bmatrix}$$

In this matrix, Z_n is the z-value of the near clipping plane. The variables w, h, and Q have the following meanings. Note that fov_w and fov_k represent the viewport's horizontal and vertical fields of view, in radians.

$$w = \cot\left(fov_w/2\right)$$

$$h = \cot\left(fov_k/2\right)$$

$$Q = \frac{Z_f}{Z_f - Z_n}$$

For your application, using field-of-view angles to define the x- and y-scaling coefficients might not be as convenient as using the viewport's horizontal and vertical dimensions (in camera space). As the math works out, the following two equations for w and h use the viewport's dimensions, and are equivalent to the preceding equations.

$$w = \frac{2 \cdot Z_n}{V_w}$$

$$h = \frac{2 \cdot Z_n}{V_h}$$

In these formulas, Z_n represents the position of the near clipping plane, and the V_w and V_h variables represent the width and height of the viewport, in camera space.

Whatever formula you decide to use, be sure to set Z_n to as large a value as possible, because z-values extremely close to the camera don't vary by much. This makes depth comparisons using 16-bit z-buffers somewhat complicated.

A w-friendly projection matrix

Direct3D can use the w-component of a vertex that has been transformed by the world, view, and projection matrices to perform depth-based calculations in depth-buffer or fog effects. Computations such as these require that your projection matrix normalize w to be equivalent to world-space z. In short, if your projection matrix includes a (3,4) coefficient that is not 1, you must scale all the coefficients by the inverse of the (3,4) coefficient to make a proper matrix. If you don't provide a compliant matrix, fog effects and depth buffering are not applied correctly.

The following illustration shows a noncompliant projection matrix, and the same matrix scaled so that eye-relative fog will be enabled.

Non-compliant

$$\begin{bmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & e \\ 0 & 0 & d & 0 \end{bmatrix}$$

Compliant

$$\begin{bmatrix} a/e & 0 & 0 & 0 \\ 0 & b/e & 0 & 0 \\ 0 & 0 & c/e & 1 \\ 0 & 0 & d/e & 0 \end{bmatrix}$$

In the preceding matrices, all variables are assumed to be nonzero. For information about w-based depth buffering, see Depth buffers.

Direct3D uses the currently set projection matrix in its w-based depth calculations. As a result, applications must set a compliant projection matrix to receive the desired w-based features, even if they do not use Direct3D for transforms.

Related topics

Transforms